Speed-Optimized Neural Indexing & Checking

Problem Statement

- Audio datasets collected from vendors needs to be checked for Plagiarism & Duplicate submissions
- Plagiarism contents cause copyright and license issues
- Any content before being used for Training needs to be checked for Plagiarism, license and duplicate violations

Proposal

- To develop a Audio Duplicate check and plagiarism check tool fast, lightweight pipeline using hash-based fingerprints + ANN search + heuristics for AI detection

< Photo >

Darshan V
Staff Engineering Mgr
darshan.venu@samsung.com
+91-9900216947

< Photo >

Ella Reddy,
Deputy Mgr
Ella.reddy@samsung.com
+91-XXXXXXXX

Additional Documentation:

- Shazam: Acoustic fingerprinting with a large database for real-time matching.
- Acoustid/Chromaprint: Open-source fingerprinting for music identification.
- YouTube Content ID: Combines fingerprinting with ML for copyright detection.
- Studies: Research on robustness of audio hashing against noise/distortions.

Work-let expected duration – 3 months

4 **Members**

Expectations

Tool to identify duplicates within the submitted batch and across datasets of audio datasets.
Tool to check for plagiarism across the internet.
Tool to check for license/copyright issues
Tool to identify duplicate and plagiarism even in case of edited files

Training/ Pre-requisites

Programming & Tools
Python (Numpy, Pandas, Scikit-learn, PyTorch/TensorFlow basics).
FFmpeg for audio handling.
Docker basics for packaging.

Audio Processing Basics
Digital audio formats, sampling, bitrates.
Librosa for feature extraction (MFCC, spectrograms).
Fundamentals of audio fingerprinting.

Machine Learning Foundations
Classification, embeddings, similarity measures.
ANN concepts (cosine similarity, nearest neighbor search).

Databases & APIs
PostgreSQL/SQLite basics.,REST APIs (FastAPI).

Deployment & Reporting
Flask/FastAPI web backend.
Simple React/HTML dashboards.
Preparing reports with visualizations.

Kick Off < 1st Month >

Activities:

- Training on audio fingerprinting & ANN- Build audio ingest pipeline (FFmpeg)- Preprocess (downsample to 16kHz mono)- Implement fast fingerprinting (Panako, Chromaprint)- Prototype ANN search (HNSWlib/MinHash)

Deliverables:

- - Preprocessing & fingerprinting prototype- ANN search prototype

Milestone 1 < 2nd Month >

Activities:

- - Optimize ANN engine for batch queries- Build compressed hash DB- Integrate with copyright hash DB- Connect to internet fast search APIs- Implement AI heuristics (pitch/artifacts)

Deliverables:

- - Fast ANN engine- Copyright matching prototype- AI heuristic module

3rd Month

Activities:

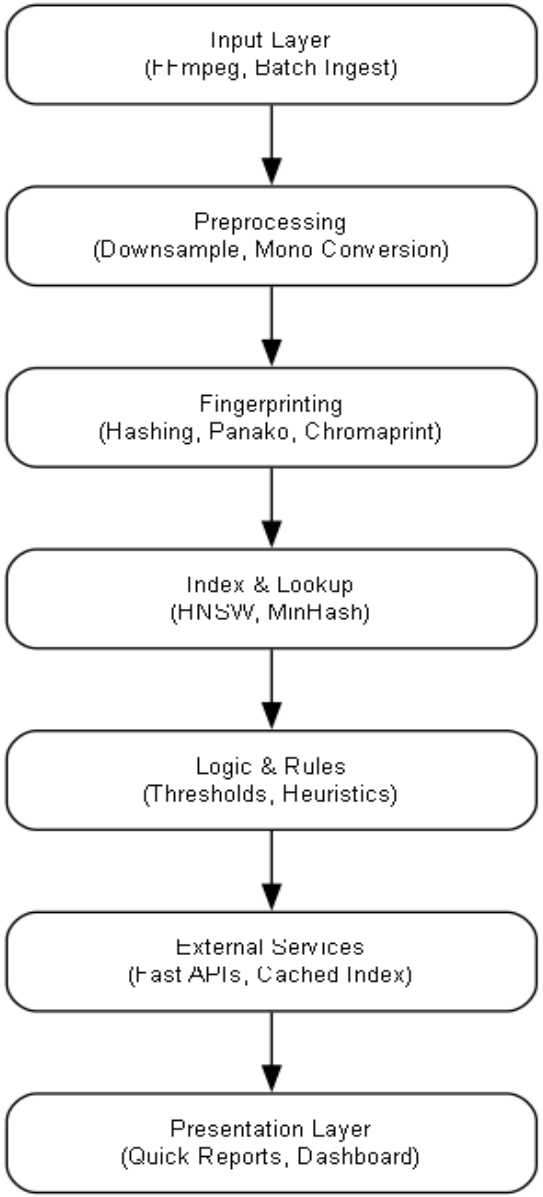
- - Develop minimal dashboard (FastAPI/React)- Integrate modules (fingerprint + ANN + heuristics)- Run speed benchmarks- Prepare quick-report outputs (CSV/JSON)- Final packaging & docs

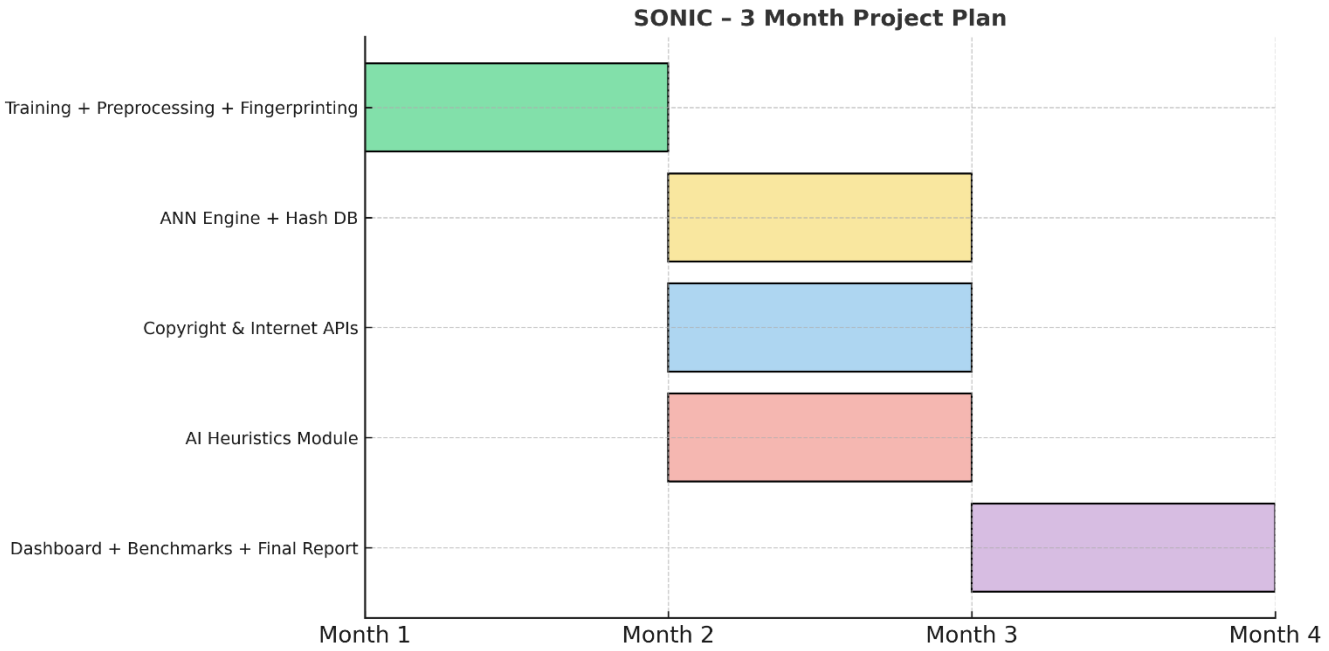
Deliverables:

- - End-to-end tool with lightweight UI- Speed benchmark results- Final project report.

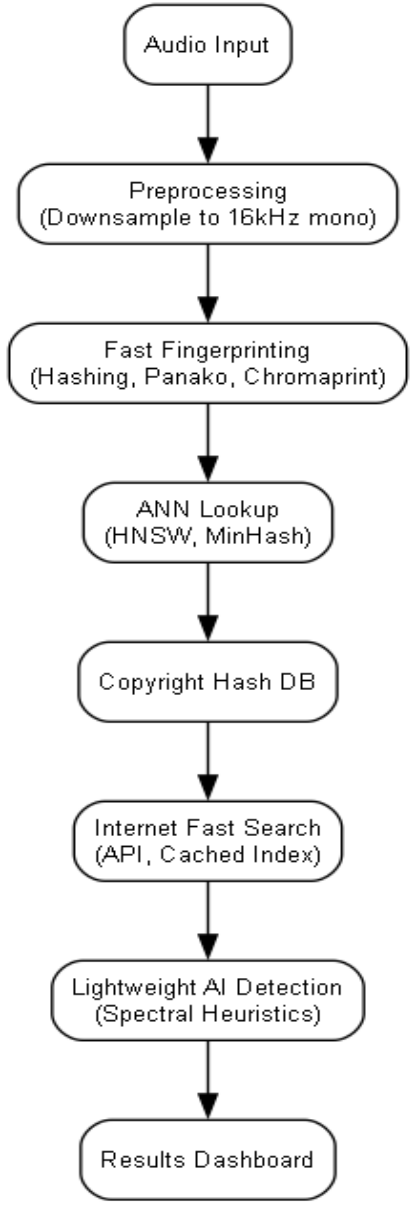
Speed-Optimized Neural Indexing & Checking

HLD





Flow Diagram



Speed-Optimized Neural Indexing & Checking

1. Input Handling
•Accepts all formats (MP3, WAV, FLAC, AAC, OGG).
•Converts via **FFmpeg**.

2. Preprocessing
•**Downsample to 16kHz mono** (low resolution but fast).
•**Normalize amplitude**.
•Skips heavy denoising — prioritizes **speed over signal purity**.

3. Fast Fingerprinting
•**Chromaprint, Panako** – generate compact fingerprints.
•**Hash-based methods (Shazam-style)** – create unique signatures from spectral peaks.
•Extremely **small storage footprint** and **fast lookups**.

4. ANN Lookup (Similarity Matching)
•Uses **Approximate Nearest Neighbor (ANN)** search instead of exact cosine/DTW.
•Libraries: **HNSWlib**, **MinHash**.
•Enables **sub-second retrieval** even in millions of fingerprints.
•Sacrifices some accuracy for speed.

5. Copyright Hash DB
•Compare fingerprints against **copyright-protected databases**.
•Direct matching ensures instant recognition of known copyrighted tracks.

6. Internet Fast Search
•Uses **cached indexes + web APIs** for duplication detection.
•Example: check if the same audio snippet exists online.
•Relies on **fast hash lookups**, not deep embeddings.

7. Lightweight AI Detection
•Instead of deep CNNs/Transformers, SONIC uses **heuristics**:
- **Spectral anomalies** (e.g., over-smoothness in TTS).
- **Pitch consistency** (synthetic voices lack natural micro-variations).
- **Artifacts** from vocoders (high-frequency unnatural bands).

•Quick but less reliable than AURA's deep classifiers.